

Ajan çerçeveleri: AutoGen, MAF, OpenClaw

Üç sistemi mekanizma mekanizma açar, aralarındaki geçiş yollarını gösterir, ve piyasadaki diğer çerçevelerle yan yana koyar. Her iddia etiketli: ölçülen, kaynaktan doğrulanan, ve yalnızca okunan ayrı ayrı işaretli.

İçindekiler

KISIM 1 • Harita

1. Üç ayrı şey, üç ayrı soru
2. İddialar nasıl etiketleniyor

KISIM 2 • AutoGen

3. Üç katman
4. Aktör modeli ve kimlik
5. İki iletişim biçimi, iki asimetri
6. AgentChat: ajan ve tool döngüsü
7. Beş takım: sırayı kim belirliyor
8. Sekiz resmî desen
9. Dört sessiz varsayılan

KISIM 3 • Microsoft Agent Framework

10. Halef: dört alan

11. Kılavuzun kendi saydığı dört fark

12. Workflow ile GraphFlow: kontrol akışı mı, veri akışı mı

13. MAF'ta ölçtüklerimiz

KISIM 4 • OpenClaw: harness katmanı

14. Harness nedir, kütüphaneden farkı ne

15. Üç kontrol eksen

16. Bellek, bağlam ve dış içerik

17. Zamanlama, dayanıklılık ve denetim

KISIM 5 • Karşılaştırma ve seçim

18. Beş çerçeve, tek tabloda

19. Ne zaman hangisi

20. Kurumsal soru: ne alınır, ne alınmaz

Harita

Üç sistem, üç ayrı soru. Karıştırıldıklarında seçim tartışması anlamsızlaşıyor, çünkü aynı yarışta değiller.

1 Üç ayrı şey, üç ayrı soru

BU BÖLÜMDE

- AutoGen, MAF ve OpenClaw hangi soruya cevap veriyor
- Neden bunlar birbirinin alternatifi değil
- Bu belgedeki etiketler ne anlama geliyor

Bir kurum “hangi ajan çerçevesini kullanalım” diye sorduğunda genellikle üç ayrı soruyu tek soruya sıkıştırmış oluyor. Ayırdığında cevap kendiliğinden çıkıyor.

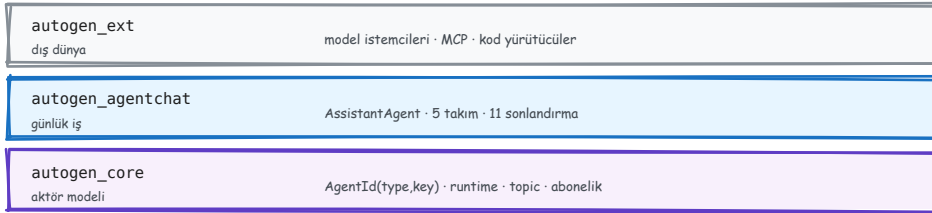
SİSTEM	NE	CEVAPLADIĞI SORU	DURUM
AutoGen	kütüphane (Python)	Birden çok ajanı nasıl koşturup birbirine bağlarım?	bakım modu · v0.7.5
MAF	kütüphane (.NET · Python · Go)	Aynı iş, kurumsal yüzeylerle: oturum, tip, ara katman, telemetri	aktif · 1.14.0
OpenClaw	harness (uçtan uca araç)	Bir ajanı gerçek bir makinede güvenle nasıl koştururum?	aktif

Kütüphane ile harness aynı yarışta değil: biri parça, öteki ürün.

AutoGen ve MAF birbirinin alternatifi — biri diğerinin **halefi**. OpenClaw ise ikisinin de alternatifi değil: onlarla kurulmuş bir sistemin etrafına geçen kontrol düzlemi. Bir kurumda üçü de olabilir, ve çoğu zaman olmalı.

NEDEN BÖYLE

Bu ayrım pratikte şuraya bağlanıyor: **kütüphane seçimi geri alınabilir, harness kararı alınamaz.** Kütüphaneyi değiştirmek ajanları yeniden yazmak demek — birkaç haftalık iş. Harness'ı değiştirmek ise onay akışını, denetim kaydını, sır yönetimini ve operasyon alışkanlıklarını değiştirmek demek.



Ayırın şey en alt katman: ajanlar gerçekten aktör — kendi mailbox'ı olan, mesajı tipe göre yönlendiren birimler.

Yukarıdan başla: aşağı inmek her zaman mümkün.

Bir ajan sisteminin katmanları. Çerçeve ortada duruyor; harness onu kuşatıyor.

2 İddialar nasıl etiketleniyor

BU BÖLÜMDE

- Üç etiket ve ne garanti ettikleri
- Neden rakip çerçeveler çoğunlukla [teyitsiz]

Bu belgedeki her iddia üç etiketten birini taşıyor. Etiket, iddianın doğruluğunu değil **dayanağını** söylüyor.

ETİKET	ANLAMI	NE GÖSTERİLEBİLİR
[ölçüldü]	Bu depoda kod koşturularak elde edildi	ölçüm dosyası ve çıktısı
[kaynak]	Birincil kaynaktan doğrulandı	resmî doküman satırı, kurulu paketin API'si, depo commit'i
[teyitsiz]	Okunandan; koşturulmadı	yalnız okunan belgenin adresi

AutoGen, MAF ve OpenClaw bu makinede gerçekten koşturuldu. LangGraph, CrewAI, OpenAI Agents SDK ve Google ADK **koşturulmadı** — onlar hakkındaki mimari cümleler **[teyitsiz]**. Bunu gizlemek belgenin geri kalanına olan güveni de düşürürdü.

TUZAK

Bir sayıyı slayta ya da rapora koyarken yanına **ölçüldüğü dosyayı** da yaz. Bu projede bir kez yapılmadı: “%63,7” sayısı doğru ölçülmüştü ama açıklaması hatırlanarak yazıldı ve yanlış şeye atfedildi. Kaynağı yazılmayan sayı, birkaç gün sonra yanlış anlatılıyor.

AutoGen

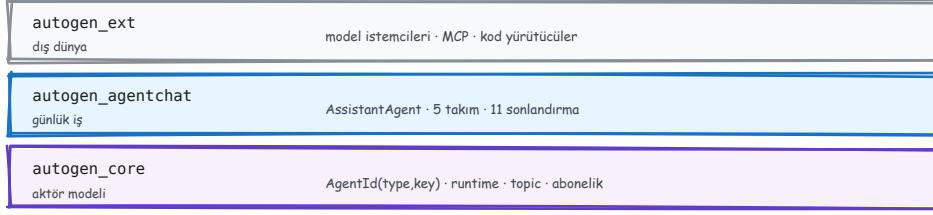
Aktör modelinden tool döngüsüne. Her mekanizmanın ne olduğu, nasıl çalıştığı, ve nerede ısırdığı.

3 Üç katman

BU BÖLÜMDE

- autogen_core · autogen_agentchat · autogen_ext ne yapıyor
- Hangi katmandan başlamalı
- Katmanları karıştırmanın maliyeti

AutoGen tek bir kütüphane değil, üst üste duran üç katman. Hangi katmanda çalıştığını bilmemek, bu çerçevede en pahalı karışıklık.



Ayırın şey en alt katman: ajanlar gerçekten aktör — kendi mailbox'ı olan, mesajı tipe göre yönlendiren birimler.
Yukarıdan başla; aşağı inmek her zaman mümkün.

Üç katman: aşağı indikçe güç artıyor, kolaylık azalıyor.

KATMAN	İÇERİĞİ	NE ZAMAN
autogen_core	aktör modeli: AgentId, runtime, topic, abonelik	gerçek eşzamanlılık, dağıtık çalışma, kendi kapını kurmak
autogen_agentchat	AssistantAgent, beş takım tipi, on bir sonlandırma koşulu	günlük iş — buradan başla
autogen_ext	model istemcileri, MCP, kod yürütücüler, üçüncü parti	dış dünyaya bağlanmak

NEDEN BÖYLE

Kural basit ve pratik: **yukarıdan başla**. AgentChat'in zaten çözdüğü bir problemi core'da yeniden çözmek, aynı işi daha az testle yapmak demek. Aşağı inmek her zaman mümkün — ve gerektiğinde iniyoruz: bu projedeki fan-in ölçümü core katmanında yazıldı.

ÖLÇÜM

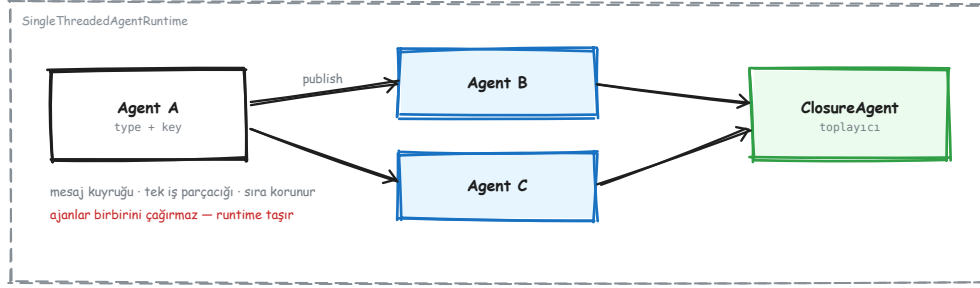
Bu projede core'a inilen tek yer `pipeline/fanin.py`. Sebebi ölçüm: GraphFlow'un birleşme bariyeri bir dal çökünce tamamlanmış kardeşleri de kaybettiriyor. Core'da kuyrukla toplayan sürüm aynı arıza altında iki sonucu ~3 ms'de topladı; GraphFlow sıfır ya da bir sonuçla süre sınırına girdi. **[ölçüldü]** — `pipeline/compare_fanin.py`

4 Aktör modeli ve kimlik

BU BÖLÜMDE

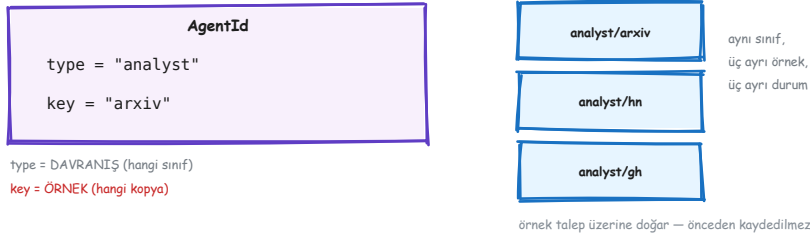
- Ajanın gerçekten ne olduğu
- AgentId'nin iki parçası ve neden ikisi de gerekli
- Topic kaynağının ajan anahtarına dönüşmesi

AutoGen'i LangGraph'tan ya da CrewAI'dan ayıran şey burada: ajanlar gerçekten **aktör** — kendi mailbox'ı olan, mesajı **type göre** yönlendiren, makinelere dağıtılabilen birimler.



Aktör modeli: bir ajan başka bir ajanın nesnesini elinde tutmuyor, runtime'a mesaj veriyor.

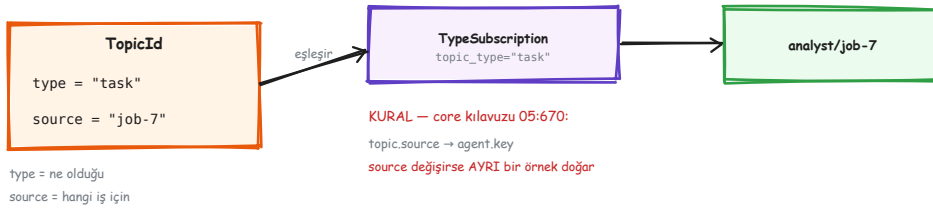
Bunun bedeli bir katman: “kim kimi çağırdı” yığın izinde görünmüyor. Karşılığında üç şey kazanılıyor — sisteme yeni ajan eklemek çağıran kodu **hiç** değiştirmiyor, bütün mesajlar tek noktadan geçtiği için müdahale ve ölçüm oraya takılıyor, ve aynı sınıftan istediğin kadar örnek doğurmak bedava.



AgentId iki parçalı: tip ne yaptığını, anahtar hangi örnek olduğunu söylüyor.

NEDEN BÖYLE

En çok işe yarayan mekanizma bu: **topic kaynağı, ajan anahtarına dönüşüyor**. TopicId("turn", "oturum-42") 'ye yayın yapmak AgentId("session", "oturum-42") ajanını yaratıyor — oturum başına izole örnek, elle sözlük tutmadan. **[kaynak]** 05:670 . Bu projedeki gateway oturumları tam olarak böyle çalışıyor.



Topic bir kanal adı değil, iki parçalı adres.

5 İki iletişim biçimi, iki asimetri

BU BÖLÜMDE

- send_message ile publish_message farkı
- Farkın adreslemede değil HATADA olduğu
- Hangisini ne zaman seçmeli

İki yol var ve aralarındaki asıl fark kaç alıcı olduğu değil — **hata olduğunda ne olduğu**.

send_message — DOĞRUDAN



- cevabı DÖNDÜRÜR
- hata **ÇAĞIRANA** fırlar
- tek alıcı, bilinen adres

publish_message — YAYIN



- None DÖNER — cevap yok
- hata yalnız **LOGLANIR**
- 0 abone de geçerli bir sonuç

Bu asimetri ölçüldü: yayında düşen bir handler sessizce düşer.

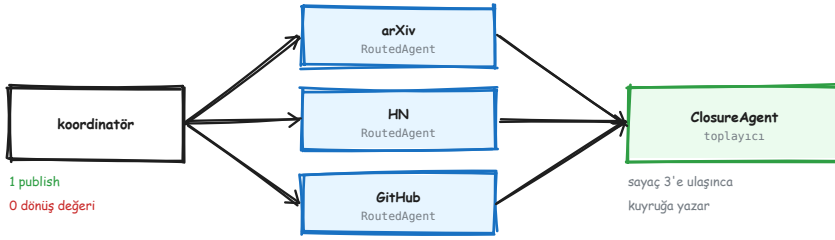
Doğrudan mesaj dönüş taşıyor ve fırlatıyor; yayın ikisini de yapmıyor.

	DOĞRUDAN (SEND_MESSAGE)	YAYIN (PUBLISH_MESSAGE)
Alıcı	tek AgentId	topic'e abone olan herkes
Dönüş değeri	var	yok
Handler çökerse	çağırana fırlatır	loglanır , fırlatmaz

Bu tek satır bir tasarım kararı: bir sonucu bekleyeceksen doğrudan, bir olayı duyuracaksan yayın. **[kaynak]** docs/14 §3.3

TUZAK

Yayının hata yutması, fan-out'ta sessiz veri kaybının kaynağı. Toplayıcı üç sonuç bekliyor, dallardan biri sessizce ölüyor, ve kimse haberdar olmuyor. Çözüm mekanik: **beklenen sonucu say**, runtime'ın “boşta” demesine güvenme.



hepsi AYNI topic'e abone — tek publish üçünü birden uyandırır

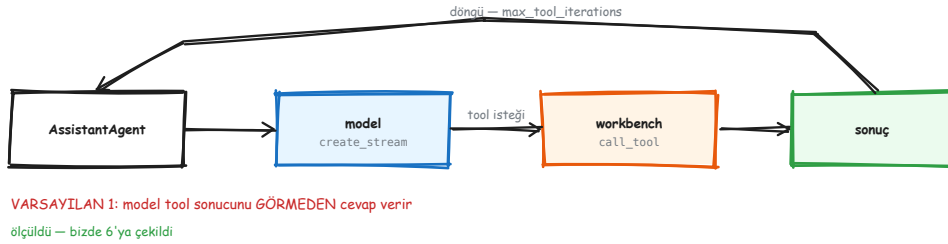
Tek yayın, üç ajan. Eşzamanlılık abonelikten geliyor, bir çağrıdan değil.

6 AgentChat: ajan ve tool döngüsü

BU BÖLÜMDE

- AssistantAgent'ın içinde ne var
- Tool döngüsünün varsayılan olarak neden bir adımda durduğu
- reflect_on_tool_use ile max_tool_iterations'ın iki ayrı anahtar olması

Günlük iş bu katmanda. **AssistantAgent** model istemcisini, tool kaynağını ve belleği tek nesnede topluyor.



Tool döngüsü: model karar veriyor, workbench çağırıyor, sonuç bağlama dönüyor.

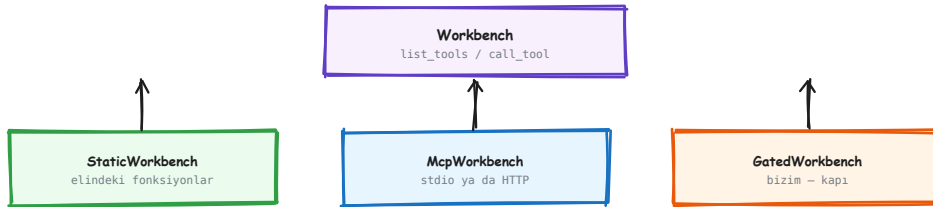
BU PROJEDEKİ KURULUM — PIPELINE/CONVERSATION.PY

```
agent = AssistantAgent(
    name="analyst",
    model_client=client,
    model_context=CompactingChatCompletionContext(...), # bellek BURADA
    workbench=[static_wb, mcp_wb], # tools= ile BİRLİKTE kullanılamaz
    model_client_stream=True,
    max_tool_iterations=6, # varsayılan 1
)
```

TUZAK

İki ayrı anahtar, iki ayrı sonuç. `max_tool_iterations` varsayılanı **1**: ajan bir tool çağırır, sonucu görür ve **durur**. Zincirleme davranış sessizce imkânsız. `reflect_on_tool_use` ise ayrı bir anahtar — kapalıyken ajan sonucu okuyup cevabı yazmıyor, ham tool çıktısı doğrudan kullanıcıya gidiyor (`ToolCallSummaryMessage`). İkişi karıştırıldığında “neden cevap yazmıyor” sorusu yanlış yerde aranıyor. **[ölçüldü]** · **[kaynak]** 08:2298

`model_context` verilmezse ajanın **belleği hiç olmuyor** ve bu hata da vermiyor — her tur sıfırdan başlıyor. Aynı şekilde `tools=` ile `workbench=` birlikte verilirse `ValueError: Tools cannot be used with a workbench.`



Tool tek bir arayüz: workbench BİR KOLEKSİYON — durum ve kaynak paylaşılan tool'lar, tek tip sonuç.

Ajan hangiyle konuştuğunu bilmiyor. Kapıyı araya koymayı mümkün kılan tek şey bu.

Workbench bir tool *kaynağı*: listeler ve çağırır. Kaynak olduğu için sarmalanabiliyor.

NEDEN BÖYLE

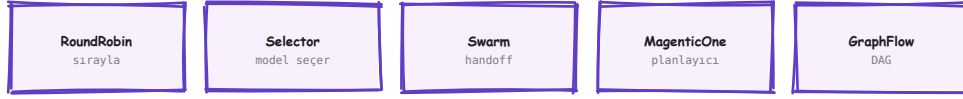
Ajana düz bir tool listesi yerine bir **kaynak** vermek, kaynağı sarmalanabilir yapıyor. Bu projedeki onay kapısı tam olarak bunu kullanıyor: `GatedWorkbench` herhangi bir workbench'i sarıp `before_tool_call` kancasını çalıştırıyor. Liste verilseydi araya girecek yer olmazdı.

7 Beş takım: sırayı kim belirliyor

BU BÖLÜMDE

- Beş takım tipinin tek ayırt edici sorusu
- Ölçülen token farkı ve neyin ödendiği
- Sonlandırma koşulu olmayan takımın maliyeti

Bir takımı diğerinden ayıran şey ajanları değil, **konuşma sırasının nereden geldiği**.



Beşi de aynı arayüz: `run()` / `run_stream()` → `TaskResult`
Taramamız `GraphFlow` kullanıyor — eşzamanlı dal + `join(all)`

Beş takım tipi ve sırayı belirleyen mekanizmaları.

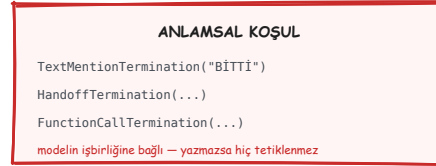
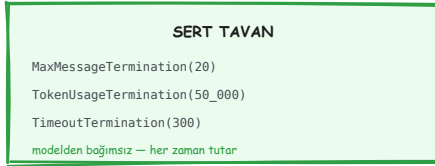
TAKIM	SIRAYI BELİRLEYEN	ÖLÇÜLEN TOKEN
SelectorGroupChat	her turdan önce bir model çağırısı	204
GraphFlow	önceden çizilmiş yönlü graf	270
RoundRobinGroupChat	sabit döngü, model karışmıyor	274
Swarm	ajanın kendi devri (tool çağırısı)	334
MagenticOneGroupChat	görev defteri tutan bir yönetici	en pahalı

Aynı görev, yalnız orkestrasyon değişiyor.

ÖLÇÜM

%63,7 fark — Selector 204, Swarm 334. Ödenen şey zekâ değil, **yönlendirme özerkliği**. Bunun karşılaştırmaya çevirisi şu: OpenAI Agents SDK'nın tek modeli olan handoff, AutoGen'in en pahalı desendir. **[ölçüldü]** · [poc/kiyas.py](#)

Bu projede beşi de gerçekten koşturuldu ve süreleri ölçüldü: GraphFlow 13 sn · Swarm 14 sn · RoundRobin 29 sn · Selector 56 sn · MagenticOne **110 sn**. **[ölçüldü]** · [pipeline/teams.py](#)



`termination = MaxMessageTermination(20) | TokenUsageTermination(50_000)`

| → biri yeterli · & → hepsi gerekli

Üretimde en az bir SERT tavan olmadan kaçmak, faturayı modelin kararına bırakmaktır.

On bir sonlandırma koşulu; birleştirilebiliyorlar.

TUZAK

Sonlandırma koşulu olmayan takım **sonsuza kadar** konuşuyor ve fatura gerçek. `MaxMessageTermination` bir üslup tercihi değil, maliyet tavanı.

TUZAK

`SelectorGroupChat` sıradaki konuşmacıyı ajanların `description` alanına bakarak seçiyor. Boşsa seçim **kör** yapılıyor — ve hata vermiyor.

`Swarm` 'ın devri özel bir tool çağırısı. Tool adı **küçük harfe düşüyor**; elle yazınca eşleşmiyor, `Handoff(target=X).name` ile üretmek gerekiyor. **[ölçüldü]**

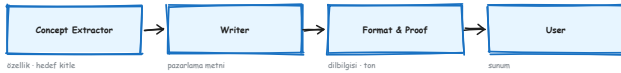
8 Sekiz resmî desen

BU BÖLÜMDE

- Kılavuzun saydığı sekiz başlık ve satır numaraları
- Hangisi orkestrasyon, hangisi yetenek
- Desenler arasındaki çelişki

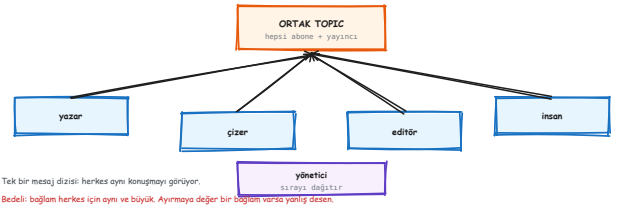
Kılavuz sekiz başlık sayıyor. Yedisi orkestrasyon deseni, sonuncusu (*Code Execution*) bir yetenek. **[kaynak]** docs/05:3206

#	DESEN	SATIR	BİR CÜMLEDE
1	Concurrent Agents	05:3236	tek yayın → çok dal → toplayıcı
2	Sequential Workflow	05:3504	her ajan bir sonrakine devrediyor
3	Group Chat	05:3772	bir yönetici konuşma sırasını dağıtıyor
4	Handoffs	05:4349	ajan işi kendisi devrediyor
5	Mixture of Agents	05:4989	aynı soru, farklı uzmanlar, birleştirici
6	Multi-Agent Debate	05:5358	birden çok tur karşılıklı eleştiri
7	Reflection	05:5822	üretici + eleştirmen, kalite döngüsü
8	Code Execution	05:6188	modelin yazdığı kod bir yürütücüde koşuyor



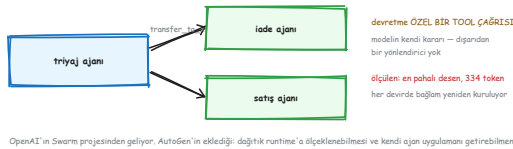
Sıra DETERMİNİSTİK: her ajan bir alt görevi yapıp bir sonrakine devrediyor. Kim konuşacak diye kimse karar vermiyor. core'da bu, her ajanın bir sonrakini 'topic'ine yayın yapmasıyla kuruyor — zincir aboneliklerde yazılı.

Sequential Workflow — sıra deterministik.



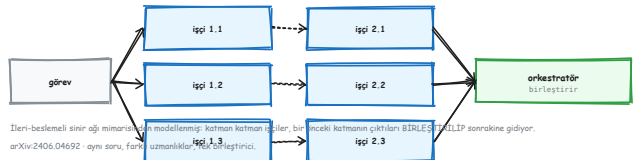
Tek bir mesaj dizisi: herkes aynı konuşmayı görüyor.
Bedeli: bağlam herkes için aynı ve büyük. Ayırmaya değer bir bağlam varsa yanlış desen.

Group Chat — tek mesaj dizisi, ortak bağlam.



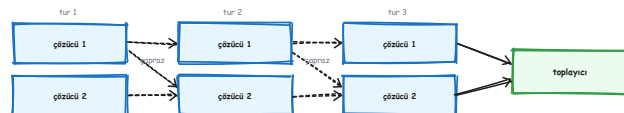
OpenAI'ın Swarm projesinden geliyor. AutoGen'in eklediği: dağıtık runtime'a ölçeklenebilmesi ve kendi ajan uygulamaları getirebilmesi.

Handoffs — devir bir tool çağırısı.



İleri-beslemeli sinir ağı mimarisinde, modellemeyi katman katman (görev, bir önceki katmanın çıktılarını BİRLEŞTİRİP sonrakine gönderiyor. arXiv:2406.04692 - aynı soru, farklı uzmanlıklar, birleştirici.

Mixture of Agents — katman katman işçiler.



Her turda ajanlar cevaplarını DEĞİŞ TOKUŞ edip birbirinin cevabına göre kendilerininkini düzeltiyor. Çözücüler seyrek bağlı — herkes herkesle değil. G5M8K matematik problemleri üstünde gösteriliyor.

Multi-Agent Debate — seyrek bağlı çözücüler.



İkinci LLM üretimi, birincinin ÇIKTISINA kopulmamış. Döngü eleştirmen tatmin olana kadar sürüyor. Bizde karşılığı: RiskAuditor — üç analizi çelişki ve kaynaksız iddia için çapraz kontrol ediyor.

Reflection — üretici ve eleştirmen.

TUZAK

Resmî desenler bu konuda **birbiriyle çelişiyor**: *Concurrent Agents* sonuçları kuyrukla topluyor, *Mixture of Agents* `asyncio.gather(...)` ile — ve sessiz kaybın kaynağı ikincisi. Kılavuzdaki bir örneği kopyalarken hangisini kopyaladığını bilmek gerekiyor.

NEDEN BÖYLE

Multi-Agent Debate kılavuzda GSM8K matematik problemleri üstünde gösteriliyor — yani **cevabı doğrulanabilen** bir alanda. Bu ipucu önemli: münazara, yanlıştın tespit edilebildiği yerlerde işe yarıyor. Görüş meselelerinde yalnızca daha uzun konuşuyorsun.

9 Dört sessiz varsayılan

BU BÖLÜMDE

- Hata vermeden yanlış sonuç üreten dört ayar
- Neden en pahalı şeyin makul görünen varsayılan olduğu

Bir çerçevenin en pahalı şeyi, makul görünen sessiz varsayılanıdır. AutoGen'de dört tane sayıldı ve dördü de sistemi **çalıştırıp** sonucu bozuyor. Hiçbiri hata vermiyor.

max_tool_iterations varsayılan: 1	model tool sonucunu GÖRMEDEN cevaplar
model_context varsayılan: yok	ajanın belleği yoktur
model_client_stream varsayılan: False	token akışı hiç yayılmaz
sonlandırma varsayılan: yok	takım tavansız koşar

Dördünün ortak yanı: **HİÇBİRİ HATA VERMEZ.**

Sistem çalışır, sonuç yanlış olur — ve aramak için önce yanlış olduğunu bilmen gerekir.

Dört sessiz varsayılan ve ısırdıkları yer.

VARSAYILAN	GÖRÜNEN DAVRANIŞ	GERÇEK SONUÇ
max_tool_iterations=1	ajan çalışıyor	zincir ilk tool'dan sonra duruyor
reflect_on_tool_use=False	cevap geliyor	gelen şey ham tool çıktısı
model_context verilmemiş	sohbet akıyor	ajanın belleği yok
stop_when_idle()	toplama bitti sanılıyor	bir dal çökünce bariyer erken açılıyor

TUZAK

Dış runtime verilmiş bir ajan çöktüğünde **fırlatmıyor, asılıyor**. Süre sınırı koymayan bir çağrı orada sonsuza kadar bekliyor.

Microsoft Agent Framework

Halef geldi. Neyi değiştirdi, neyi henüz vermiyor, ve geçiş nasıl yapılıyor — satıcının kendi kılavuzundan.

10 Halef: dört alan

BU BÖLÜMDE

- MAF'ın kendi tanımladığı dört ana alan
- AutoGen ve Semantic Kernel ile ilişkisi
- Hangi dillerde ne kadar hazır

Microsoft Agent Framework, AutoGen ile Semantic Kernel'in **ikisinin birden** halefi. Kılavuzun kendi cümlesi: *“The Agent Framework is the direct successor, created by the same teams... In short, Agent Framework is the next generation of both Semantic Kernel and AutoGen.”* **[kaynak]**

ALAN	İÇERİĞİ
Agents	LLM'i çağıran, tool ve MCP sunucusu kullanan tekil ajanlar
Harness Agent	“batteries-included” ajan: planlama ve todo takibi, bağlam sıkıştırma, dosya erişimi ve bellek, bir daha sorma tool onayı, gözlemlenebilirlik
Workflows	ajanları ve fonksiyonları açık yürütme yollarıyla bağlayan graf tabanlı akışlar
Integrations	model sağlayıcıları, ajan servisleri, tool'lar, bağlam sağlayıcıları, ara katman, değerlendirme, arayüz

MAF'ın kendi ana sayfasındaki dörtlü. **[kaynak]**

Bunların altında temel yapı taşları duruyor: model istemcileri, durum için **agent session**, bellek için **context providers**, araya girmek için **middleware**, ve tool entegrasyonu için MCP istemcileri.

NEDEN BÖYLE

Harness Agent bu listede en dikkat çekici madde. İçindekiler tanıdık gelmeli: planlama, todo takibi, bağlam sıkıştırma, dosya erişimi, tool onayı, gözlemlenebilirlik. Bunlar bir *harness*'in işleri — yani OpenClaw'ın yaptığı şey. Halef çerçeve, harness katmanının bir kısmını **kütüphanenin içine** alıyor.

TUZAK

Diller eşit değil. Go sürümü **public preview** ve kılavuzun kendi uyarısına göre *declarative agents*, *RAG*, *CodeAct* ve *functional workflows* henüz yok. **[kaynak]**

11 Kılavuzun kendi saydığı dört fark

BU BÖLÜMDE

- Microsoft'un AutoGen → MAF geçiş kılavuzundaki dört başlık
- Hangisinin ilerleme, hangisinin geri adım olduğu

Geçiş kılavuzu farkları dört maddede topluyor. Alıntılar birebir. **[kaynak]**

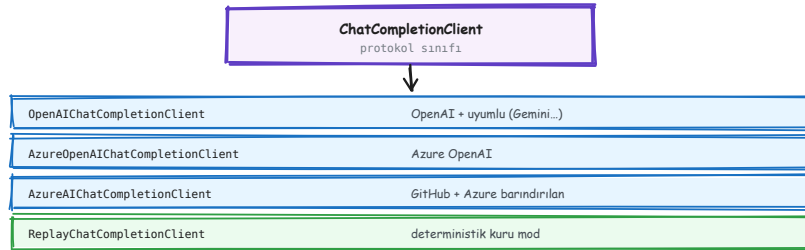
#	KONU	AUTOGEN	MAF
1	Orkestrasyon	olay güdümlü core + üst düzey Team	tipli, graf tabanlı Workflow ; kenarlarda veri akıyor, girdiler hazır olunca executor tetikleniyor
2	Tool'lar	FunctionTool ile sarmalama	@tool , şemayı otomatik çıkarıyor; ayrıca hosted tool'lar (kod yorumlayıcı, web arama)
3	Ajan davranışı	AssistantAgent tek turlu — max_tool_iterations artırılmadıkça	Agent varsayılan olarak çok turlu ; nihai cevabı verene kadar tool çağırmaı sürdürüyor
4	Runtime	gömülü + deneysel dağıtık runtime	bugün tek süreç ; dağıtık çalışma planlanan

ÖLÇÜM

Üçüncü madde bu projede bağımsız olarak ölçülmüştü: **max_tool_iterations** varsayılanı 1 ve zincir bir adım sonra duruyor. Satıcının kendi kılavuzu aynı şeyi söylüyor ve halefte varsayılanı **tersine çevirmiş**. **[ölçüldü]** + **[kaynak]**

TUZAK

Dördüncü madde bir **geri adım**: AutoGen'de dağıtık runtime (deneysel de olsa) var; MAF bugün tek süreç. Birden çok makineye yayılmayı planlayan bir mimari için bu, halefe geçişin bedeli.



Hepsi aynı protokolü uyguluyor — ajan hangisiyle konuştuğunu bilmiyor. Kuru mod istemcisi de bir istemci.

Model istemcisi katmanı — iki çerçevede de aynı rolü oynuyor, adı ve seçenek sistemi değişiyor.

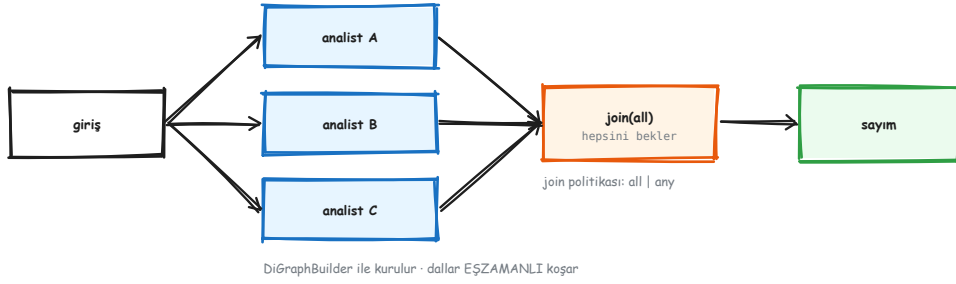
12 Workflow ile GraphFlow: kontrol akışı mı, veri akışı mı

BU BÖLÜMDE

- İki grafın kenarlarının farklı şeyler taşıması
- Bu farkın pratikte neyi değiştirdiği
- Duraklama ve checkpoint

Bu, iki çerçeve arasındaki **en derin** fark, ve kılavuz onu açıkça yazıyor. **[kaynak]**

	AUTOGEN GRAPHFLOW	MAF WORKFLOW
Temel	kontrol akışı	veri akışı
Kenar ne demek	geçiş (transition)	tipli veri yolu
Mesaj nasıl gidiyor	herkese yayın ; koşullar yayınlanan içeriğe bakıyor	belirli kenarlardan yönlendiriliyor
Düğüm ne	ajan	executor: ajan, fonksiyon ya da alt-akış
Duraklama	yok	request/response ile duraklayabiliyor
Checkpoint	yok	var



AutoGen'de graf sırayı belirliyor; mesaj zaten herkese gitmiş oluyor.

NEDEN BÖYLE

Pratikteki sonuç şu: AutoGen'de **her ajan her mesajı görüyor**. Bütün katılımcılar tek bir paylaşılan `group_topic_type` 'a abone; graf yalnız sıranın kimde olduğunu söylüyor. Bağlam herkes için aynı ve büyük — ayırmaya değer bir bağlam varsa bu yanlış yapı. MAF'ta kenar bir veri yolu, dolayısıyla bir executor yalnız kendisine gönderileni görüyor.

TUZAK

Bu fark maliyete doğrudan yansıyor. Yayın modelinde bir ajan eklemek, **bütün** konuşmayı bir kez daha modele göndermek demek. Veri akışı modelinde yalnız o kenardan geçen taşınıyor.

13 MAF'ta ölçtüklerimiz

BU BÖLÜMDE

- Kurulu paketten doğrulanan API farkları
- Onayın çerçeveye gömülü hâli
- Ortak tuzak: tool sonrası boş cevap

Aşağıdakiler bu makinede `agent-framework 1.14.0` kurulup koşturularak elde edildi.
[ölçüldü]

KONU	AUTOGEN	MAF 1.14.0
Onay	yok — <code>GatedWorkbench</code> 'i biz yazdık	<code>FunctionTool(approval_mode=...)</code> — tool'un kendi alanı
Çağrı tavanı	<code>max_tool_iterations</code> , ajanın tamamına	<code>max_invocations</code> , tool başına
Reddin dönüşü	hata işaretli <code>ToolResult</code> (elle kurduk)	<code>AgentResponse.user_input_requests</code> — cevabın birinci sınıf alanı
Akış	<code>run()</code> ve <code>run_stream()</code> iki ayrı metot	<code>run(stream=True)</code> — tek metot, parametre
Ara katman	yok	<code>ToolApprovalMiddleware</code> hazır

```

tool = af.FunctionTool(
    func=sirket_sayisi,
    approval_mode="always_require",    # AutoGen'de karşılığı YOK
    max_invocations=2,                # tool başına tavan
)
agent = af.Agent(client=client, tools=[tool],
    middleware=[af.ToolApprovalMiddleware(...)])
result = await agent.run(soru, session=af.AgentSession(session_id="x"))

```

ÖLÇÜM

`approval_mode="always_require"` ile koşulduğunda tur **duruyor**: `finish_reason="tool_calls"` ve cevap yerine bir `function_approval_request` dönüyor. Onay, AutoGen'de bizim elle kurduğumuz bir sonuç tipi; MAF'ta cevabın kendi alanı.

TUZAK

İki ölçülmüş sürpriz. Birincisi: `ToolApprovalMiddleware` bir `AgentSession` olmadan `RuntimeError` atıyor — onay, oturumu olan bir koşuya bağlı.

İkincisi ve daha ilginç: **tool çağrıldığında** `response.text` **boş kalıyor**, cevap `messages` içinde. Bu, AutoGen'in `reflect_on_tool_use=False` varsayılınının tıpatıp aynı sonucu. **İki çerçeve de tool sonrası nihai cevabı varsayılan olarak yazdırmıyor** — yalnız `text` 'e bakan bir arayüz, tool kullanan her turda boş ekran gösteriyor.

NEDEN BÖYLE

İki çerçeveyi aynı ortamda kurmayı denedik ve **olmadı**: pip'in bağımlılık çözücüsü on dakikada karar veremedi. Bu projede MAF ayrı bir sanal ortamda alt süreç olarak koşuyor (`pipeline/maf_runner.py`). Yan yana çalıştırmayı planlıyorsan bunu baştan hesaba kat. **[Ölçüldü]**

OpenClaw: harness katmanı

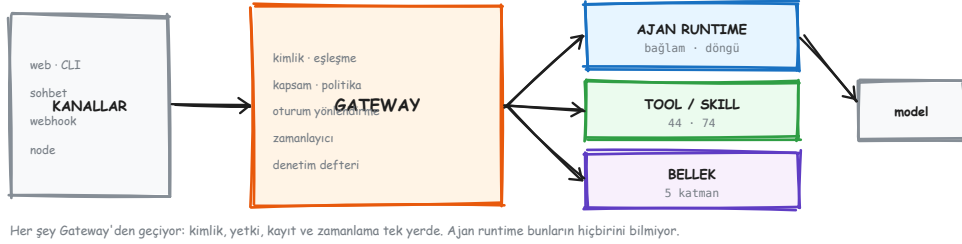
Kütüphane değil, uçtan uca bir araç. Bir ajanı gerçek bir makinede koştururken ortaya çıkan soruların cevapları burada.

14 Harness nedir, kütüphaneden farkı ne

BU BÖLÜMDE

- Harness'ın kapsadığı işler
- Neden bu işler kütüphanede olmuyor
- OpenClaw'ın mimarisi kuşbakışı

Bir kütüphane “ajanları nasıl bağlarım” sorusunu cevaplıyor. Bir **harness** ise “bu ajanı gerçek bir makinede, gerçek dosyalarla, gerçek sırlarla, gerçek bir insanın yanında nasıl koştururum” sorusunu.



OpenClaw kuşbakışı: gateway, oturumlar, tool'lar, zamanlayıcı ve denetim.

HARNESS'IN İŞİ	AUTOGEN'DE	MAF'TA
Tool onayı	yok — elle kurulur	approval_mode · ToolApprovalMiddleware
Bağlam sıkıştırma	yok	Harness Agent içinde
Dosya erişimi ve bellek	Memory protokolü (kurulum sende)	context providers · Harness Agent
Zamanlama	yok	yok
Sır yönetimi	yok	yok
Denetim kaydı	olay akışı var, kayıt yolu sende	telemetry var, uyum kaydı sende

Harness'ın kapsadığı işlerin çoğu iki kütüphanede de yok.

NEDEN BÖYLE

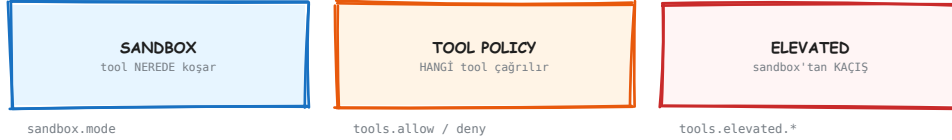
Bu tablo, MAF'ın **Harness Agent**'i neden eklediğini de açıklıyor: kütüphane katmanı, harness işlerinin bir kısmını içeri almaya başladı. Ama zamanlama ve sır yönetimi hâlâ dışarıda — ve bir kurumda bu ikisi çoğu zaman en sert gereksinim.

15 Üç kontrol eksenı

BU BÖLÜMDE

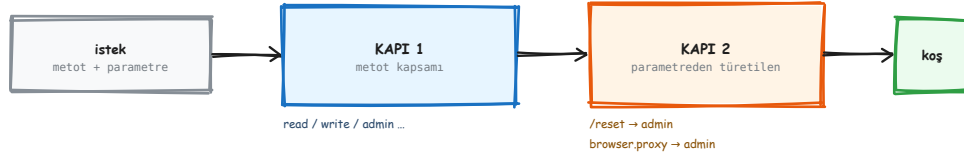
- Yetkinin üç ayrı eksende ayrılması
- Metot kapsamının neden yalnızca ilk kapı olduğu
- Donmuş plan: onay komuta değil plana bağlanır

OpenClaw'ın en taşınabilir fikri bu: yetki tek bir “izin var/yok” anahtarı değil, **üç ayrı eksen**.



deny HER ZAMAN kazanır - allow doluyorsa gerisi kapalı
Ama: tool policy ADA göre filtreler — exec'in İÇİNİ görmez.
"write"ı kapattık, artık read-only" YANLIŞTIR.

Üç eksen: kim, ne, nerede — ayrı ayrı daraltılıyor.



Yetki yükseltme yasak: bir cihazı onaylamak yalnız çağırmanın ZATEN sahip olduğu kapsamı basabilir.
Ve sessiz genişleme yok — daha geniş rol isteyen yeniden bağlanma, yeni bir bekleyen talep doğurur.

Metot kapsamı ilk kapı; arkasında ikinci bir katman var.

TUZAK

Metot kapsamı **yalnızca ilk kapı**. Bir metoda izin vermek, o metodun ulaşabileceği her şeye izin vermek demek değil — ama kapsamı tek savunma sanmak tam olarak bu hataya yol açıyor.



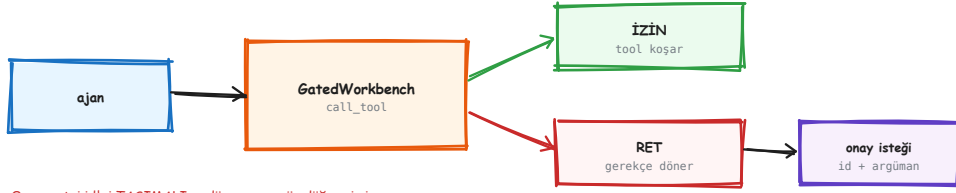
Dosya onaydan sonra değişirse de reddedilir — kaymış içerik koşmaz.
Bizim approval.py bugün argüman hash'i TUTMUYOR. Açık iş.

Donmuş plan: onaylanan şey komut değil, çalışacak planın kendisi.

ÖLÇÜM

Bu fikir bu projede ölçülerek doğrulandı. Onayı **çözülmüş** zamanlama üstünde imzaladığımızda onay **hiç tüketilemedi**: "5dk sonra" her ayrıştırmada farklı bir zaman damgasına çözülüyor, yani her seferinde yeni bir istek doğuyor. İmzayı kullanıcının yazdığı satıra taşıyınca düzeldi. **[ölçüldü]**

Aynısı kod yürütmeye de yaşandı: model aynı soruya her seferinde **farklı bir program** yazıyor. Onayı yeniden üretilen koda bağlarsan hiç tüketilemiyor; onaylanan **metnin** saklanması gerekiyor. **[ölçüldü]**



Onay metni id'yi TAŞIMALI — düşerse arayüz düğmeyi çizemez.

Bu hatayı ölçtük: testi önce eski koda karşı düşürdük.

Kapı: her tool çağrısının geçtiği tek nokta.

16 Bellek, bağlam ve dış içerik

BU BÖLÜMDE

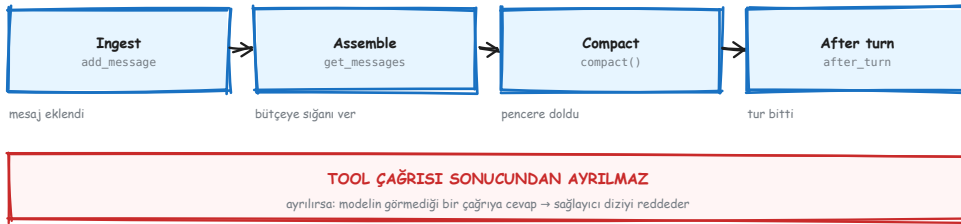
- Beş bellek katmanı ve hangisinin ne zaman okunduğu
- Bağlam motoru ve sıkıştırmanın bedeli
- Dış içeriğin veri olarak işaretlenmesi

KATMAN	YÜZEY	KİM YAZAR	NE ZAMAN ENJEKTE
Instructions	AGENTS.md	yalnız insan yazar	her oturum
Curated core	MEMORY.md	kapılı konsolidasyon	her oturum
Episodic	memory/GG.md	ajan çalışırken	hiç — aranır
Prospective	SQLite	intent tool	tetiklenince
Review	DREAMS.md	dreaming	hiç — insan okur

Sınır ikinci ile üçüncü arasında: episodic'ten curated'a hiçbir şey kapıdan geçmeden çıkmaz.

Beş bellek katmanı — hepsi aynı anda okunmuyor.

Belleğin katmanlara ayrılmasının sebebi maliyet: her turda her şeyi göndermek, en kısa soruyu bile tam tarifeden ödetiyor.



Bağlam motoru: bütçeye göre seçim, aşınca özet.

TUZAK

Sıkıştırma bir **kayıptır**. Sıkıştırma sonrası ajan, konuşmanın erken kısmındaki bir ayrıntıyı artık bilmiyor olabilir — ve bilmediğini de bilmiyor. Bir denetim sorusunun cevabı o ayrıntıdaysa, cevap sessizce yanlış oluyor.



<<<EXTERNAL_UNTRUSTED_CONTENT id="a3f9...">>>

id rastgele ÇÜNKÜ sabit olsaydı içerik kendi kapanışını yazıp sarmalayıcıdan çıkardı.

Dış içerik veri olarak işaretleniyor, talimat olarak değil.

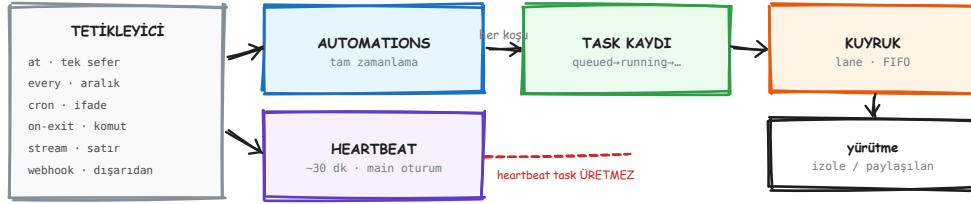
NEDEN BÖYLE

Bir web sayfasından ya da e-postadan gelen metin, modelin gözünde kullanıcının yazdığı metinle aynı görünüyor. Sarmalayıcı bu ikisini ayırıyor. Bu, MagenticOne kılavuzunun da açıkça uyardığı yer: *"prompt injection attacks from webpages"*. **[kaynak]**

17 Zamanlama, dayanıklılık ve denetim

BU BÖLÜMDE

- Zamanlayıcının gerçekte nerede koştuğu
- Dayanıklı durum ile dayanıklı yürütmenin farkı
- Neden iki ayrı kayıt hattı gerekiyor



Sıra soldan sağa: ne tetikler → kim karar verir → ne kaydedilir → nasıl serileşir → nerede koşar.

Zamanlama yığını: kayıt, tetikleyici, ve koşan süreç.

TUZAK

Zamanlama yalnız gateway **ayaktayken** çalışıyor. Bu makinede servis systemd kullanıcı servisi ve `Linger=no` — oturum kapanınca zamanlama da duruyor. “Zamanlama var” demekle “her zaman çalışır” demek arasındaki fark burada. **[ölçüldü]**

DURUM	DEPOLAMA	YENİDEN BAŞLATMADA
konuşma geçmişi	ajan SQLite	dokunulmaz
yarıda kalan tur	oturum satırı	otomatik devam
subagent koşuları	paylaşılan SQLite	boot'ta geri yüklenir
zamanlanmış işler	paylaşılan SQLite	tanım + geçmiş korunur
giden mesajlar	SQLite	drenaj edilir

① Gecikmiş işler YENİDEN ZAMANLANIR, anında oynatılmaz — soğuk başlangıçta yük tepesi olmasın.

② Kurtarma sınırlı: tekrar tekrar düşen oturum karantinaya alınır.

③ Ulaştırma iki kanıtı: önce runtime, 5 dk grace sonrası dayanıklı koşu geçmişi.

Dayanıklı durum var; dayanıklı yürütme yok.

NEDEN BÖYLE

“Dayanıklı” kelimesi iki farklı şeyi anlatıyor. OpenClaw'da dayanıklı **durum** var — her şey SQLite'ta. Ama dayanıklı **yürütme** yok: kurtarma bir replay değil, modele yazılan sentetik bir cümle. Yan etkili bir tool çağırıldıktan sonra çökülürse, ikinci kez çağırılmasını mekanik olarak engelleyen hiçbir şey yok. **[kaynak]** docs/18



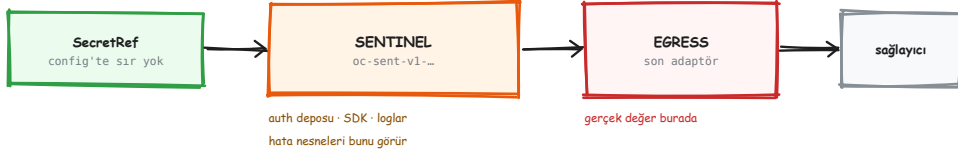
OpenClaw yalnız solu yapıyor ve bunu açıkça söylüyor:

"Bir satırın yokluğu hiçbir şey kanıtlamaz."

KKB'nin ihtiyacı sağ taraf. Ayrımı baştan yapmak ucuz, sonradan şema göçü.

İki kayıt hattı: operasyonel ve uyum.

Operasyonel kayıt **kayıp toleranslı** olmalı — kayıt uğruna işi durdurmak saçma. Uyum kaydı ise **kayıpsız, senkron ve fail-closed** olmalı: yazılamıyorsa koşu düşmeli. Aynı hattan ikisini birden beklemek, ikisinden de olmak demek.



Bilinmeyen sentinel-şekilli değer **AÖ ETKİNLİĞİNDEN ÖNCE** fail-closed: çözülmemişi iletmektense istek gönderilmez.

Ve sınır beyanı: "Sentinel'ler süreç izolasyonu DEĞİLDİR" — gerçek değer aynı süreçte bellekte.

Sır yalnız son sınırda gerçek değerine dönüşüyor.

Karşılaştırma ve seçim

Beş çerçeve yan yana, ve “hangisi” sorusunun işe yarar hâli:
hangi kısıt altında hangisi.

18 Beş çerçeve, tek tabloda

BU BÖLÜMDE

- Her çerçevenin merkezî metaforu
- Eksen eksen karşılaştırma
- Hangi iddiaların koşturularak doğrulandığı

Aşağıdaki tabloda **AutoGen, MAF ve Google ADK sütunları [kaynak]** — koşturuldu ya da birincil kaynaktan doğrulandı. Diğerleri **[teyitsiz]**.

ÇERÇEVE	MERKEZİ METAFOR	TEMEL YAPI TAŞLARI			
AutoGen	Aktör	<code>AgentId(type, key)</code> · runtime · topic · abonelik			
MAF	Tipli akış	<code>Agent</code> · <code>Workflow</code> · <code>executor</code> · <code>session</code>			
LangGraph	Graf	düğüm + kenar + state; checkpointer			
CrewAI	İnsan ekibi	<code>Agent</code> · <code>Task</code> · <code>Crew</code> (rol/backstory)			
OpenAI Agents SDK	Devir	<code>Agent</code> · <code>Tool</code> · <code>Handoff</code> · <code>Guardrail</code>			
Google ADK	Derlenen graf	<code>Agent</code> + <code>Workflow</code> , <code>validate_graph()</code>			
EKSEN	AUTOGEN	MAF	LANGGRAPH	AGENTS SDK	ADK
iletişim	pub/sub ortak thread	tipli kenarlar	paylaşılan state	yalnız handoff	graf kenarları
akış kararı	konusmacı seçimi	kenarlar + hazır girdi	kenarlar	ajanın kendisi	derlenen kenarlar
hata ne zaman	çalışma zamanı	çalışma zamanı	çalışma zamanı	çalışma zamanı	graf kurulurken
runtime	aktör, dağıtılabilir	tek süreç (bugün)	graf yürütücü	tur döngüsü	graf motoru
durability	zayıf	checkpoint	checkpointer	session	<code>Session</code>
desen çeşitliliği	5 takım tipi	workflow + hazır kalıplar	supervisor/swarm	tek	graf + Task
yaşam döngüsü	bakım modu	aktif	aktif	aktif	aktif

TUZAK

İsim karışıklığı. “AutoGen” dört ayrı şeye işaret ediyor: microsoft/autogen v0.4+ (bu belgedeki), terk edilmiş v0.2 (ConversableAgent , initiate_chat), ag2ai/ag2 forku, ve halef MAF. Bir kaynakta ConversableAgent görüyorsan o kaynak v0.2 ya da AG2 anlatıyor ve buradaki hiçbir şeyle uyumlu değil. **[kaynak]**

NEDEN BÖYLE

Graf konusunda taraflar yer değiştirdi. AutoGen grafi GraphFlow olarak beş takım tipinden *biri* diye ekledi; ADK grafi **merkeze** aldı; MAF ise grafi veri akışına çevirdi. Yani “graf mı konuşma mı” sorusu üç çerçevede üç farklı cevap alıyor.

19 Ne zaman hangisi

BU BÖLÜMDE

- Kısıta göre seçim tablosu
- “Ajan gerekmiyor” cevabının ne zaman doğru olduğu

“Hangisi daha iyi” sorusunun cevabı yok; “**bu kısıt altında hangisi**” sorusunun var.

KISIT / İHTİYAÇ	ÖNERİ	NEDEN
Yeni bir proje, Microsoft dünyası	MAF	AutoGen bakım modunda; halef aktif ve aynı ekipten
Mevcut AutoGen kodu, çalışıyor	AutoGen'de kal, gömerek	Geçişin bedeli var; ince bir arayüzün arkasına al, sonra taşı
Gerçek eşzamanlılık, çok makine	AutoGen (bugün)	MAF tek süreç; dağıtık planlanan ama yok [kaynak]
Akış önceden belli, öngörülebilirlik şart	ADK ya da LangGraph	Graf derleme zamanında doğrulanıyor; AutoGen'de hata çalışma zamanında
Uzun süren iş, kaldığı yerden devam	MAF ya da LangGraph	Checkpoint ve duraklama birinci sınıf
En düşük öğrenme eğrisi, tek devir modeli	Agents SDK	Tek desen; ama ölçülen en pahalı desen de o [ölçüldü]
Gerçek makinede, insan gözetiminde koşacak	harness (OpenClaw gibi)	Onay, sır, zamanlama ve denetim kütüphanede yok
İş bir fonksiyonla yapılabiliyor	ajan kullanma	MAF kılavuzunun kendi cümlesi [kaynak]

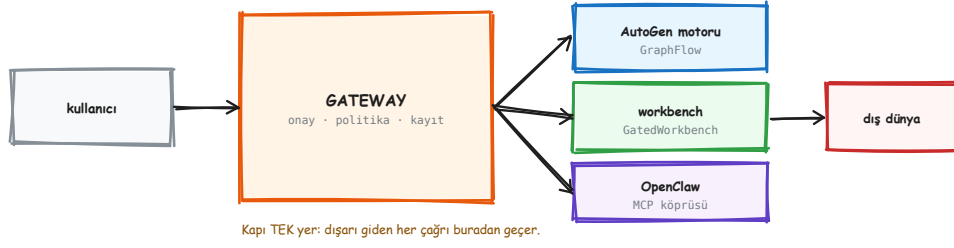
NEDEN BÖYLE

Son satır ciddi bir öneri ve kaynağı satıcının kendisi: “*If you can write a function to handle the task, do that instead of using an AI agent.*” Bir ajan, belirsizlik olmayan bir işte yalnızca pahalı ve öngörülemez bir `if` zinciridir.

20 Kurumsal soru: ne alınır, ne alınmaz

BU BÖLÜMDE

- Mekanizmanın taşınabilir, güven modelinin taşınamaz olması
- Bir kurumun kütüphaneye ekleme yapması gereken yerler
- Karar özeti



Kurumsal bir asistanda çerçeve ile kontrol düzleminin ayrılması.

Bu belgede anlatılan her **mekanizma** kurumsal bir asistana taşınabilir: kapı, donmuş plan, köken sınıfı, iki kayıt hattı, üç eksen. Taşınamayan tek şey **güven modeli**.

TUZAK

OpenClaw **tek bir güvenilen operatörün** etrafında tasarlanmış. Belgelerindeki bütün “bu bir güvenlik sınırı değildir” cümleleri buradan geliyor: o modelde zaten herkes güvenilir, dolayısıyla ayrımlar bir kolaylıktan ibaret. Birbirine güvenmeyen departmanların olduğu bir kurumda ise **aynı cümlelerin her biri bir açık**.

KURUMUN EKLEMESİ GEREKEN

Fail-closed uyum kaydı

Kimlik ve yetki (kim, ne, nerede)

Sır sınırı

Yan etki tekrarı koruması

Dış içerik sarmalayıcısı

NEDEN KÜTÜPHANEDE YOK

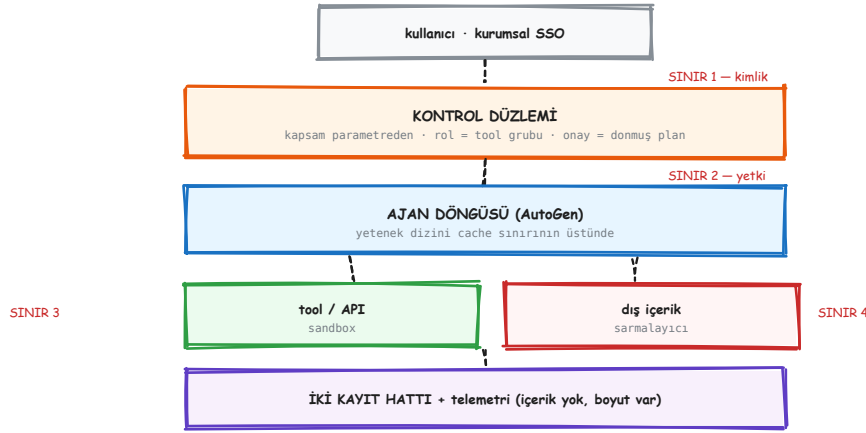
Kütüphane operasyonel kayda göre tasarlanmış

Kütüphane tek operatör varsayıyor

Model istemcisine anahtar veriliyor, ötesi sende

Dayanıklı yürütme iki çerçevede de yok

Prompt injection savunması uygulama katmanında



Kurumsal asistan: çerçeve içeride, kontrol düzlemi dışarıda.

NEDEN BÖYLE

Karar üç cümlede özetlenebiliyor. **Çerçeveyi göm** — ince bir arayüzün arkasına al, çünkü AutoGen bakım modunda ve halefe geçiş bir gün gelecek. **Harness'tan öğren** — mekanizmaları al, TypeScript kodunu değil. **Güven modelini yeniden kur** — taşınamayan tek parça o, ve kurumsal riskin tamamı orada.